Vehicle Event Logging and Response System

Case Study Report

STM32F411CEU6 (ARM Cortex-M4) – Bare-Metal Embedded C

Academic Details and Acknowledgement

Student Name(s) and Registration Number(s):

Registration No.	Name
CB.SC.U4CSE24502	Aniruth Karthik
CB.SC.U4CSE24513	Jaikrrish S
CB.SC.U4CSE24521	Krishanth K
CB.SC.U4CSE24550	Srisanth P A

Mentor: Dr. A. Baskar

Course: 23CSE304 – Embedded System

Department: Department of Computer Science and Engineering

Institution: School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India.

Contents

1 Abstract **3**

2 Introduction and Problem Statement **3**

3 Objectives and Scope **3**

3.1 Objectives . . . 3

3.2 In Scope (as implemented) . . . 3

4 System Overview and Architecture **4**

4.1 Block Diagram . . . 4

4.2 Processing Pipeline . . . 4

4.3 Main-Loop Timing . . . 4

5 Hardware Design **5**

5.1 Components . . . 5

5.2 Pin Map (from `config.h`) . . . 5

5.3 Clocking . . . 5

6 Embedded Concepts Applied **6**

7 Firmware Design **6**

7.1 Clock and Timing . . . 6

7.2 LM35 Temperature . . . 6

7.3 HC-SR04 Distance . . . 6

7.4 Bit-Banged I^2C . . . 7

7.5 MPU6050 Driver . . . 7

7.6 Emergency Interrupt . . . 7

7.7 UART Driver . . . 7

7.8 Event Manager . . . 8

7.9 Debug Observability . . . 8

8 Event Classification Logic **8**

9 UART Alert Protocol **9**

10 Testing and Validation **9**

10.1 Standalone Test Programs . . . 9

10.2 Test Plan (from the PRD) . . . 10

10.3 Recorded Results . . . 10

11 Implementation vs. Planned Design **10**

12 Future Work **11**

13 Repository Structure **11**

14 Build, Flash and Run **11**

15 Conclusion **12**

1 Abstract

This project implements a **Vehicle Event Logging and Response System** on an STM32F411CEU6 "Black Pill" microcontroller. Three sensors are monitored continuously: an **MPU6050** accelerometer/gyroscope (impact and motion, over I^2C), an **LM35** analog temperature sensor (over the ADC), and an **HC-SR04** ultrasonic ranger (obstacle distance, using GPIO and cycle-accurate timing). Readings are classified in firmware as *Normal*, *Warning*, *Critical* or *Invalid* against configurable thresholds. Critical events are transmitted as structured ASCII alerts over UART at 115200 baud and are also indicated locally by an LED. The firmware is written at the **register level** using only CMSIS device headers; it uses no HAL and no RTOS.

2 Introduction and Problem Statement

A vehicle can suffer sudden impact, overheating, or close-range obstacles with no automated mechanism to detect, record or report the event. Without continuous monitoring, such events may go unnoticed or be reported too late.

This system addresses that gap by:

- continuously sampling vehicle-mounted sensors,
- detecting abnormal conditions by comparing readings against defined thresholds,
- giving an immediate local indication (LED), and
- sending a structured alert message to a PC terminal, which stands in for notification of emergency services.

Demonstration note: Alerts are received on a PC serial terminal (Docklight / PuTTY / CoolTerm) through a USB-to-UART adapter. `[ALERT]` messages represent the emergency-notification channel.

3 Objectives and Scope

3.1 Objectives

#	Objective	Realisation
O1	Temperature monitoring	LM35 + ADC1, 32-sample averaging, over-temperature alert
O2	Impact / motion detection	MPU6050 over I^2C, baseline subtraction, threshold on peak axis deviation
O3	Obstacle detection	HC-SR04 trigger/echo timing, distance in cm, proximity alert
O4	Local response	PC13 LED: 1 Hz heartbeat when normal, 10 Hz strobe when critical
O5	Remote notification	Structured ASCII alerts and telemetry over UART at 115200 8-N-1
O6	Asynchronous emergency input	EXTI interrupt on PB12 (manual crash/SOS trigger)
O7	Register-level implementation	No HAL; direct access to RCC, GPIO, ADC, USART, EXTI, SYSCFG and DWT registers

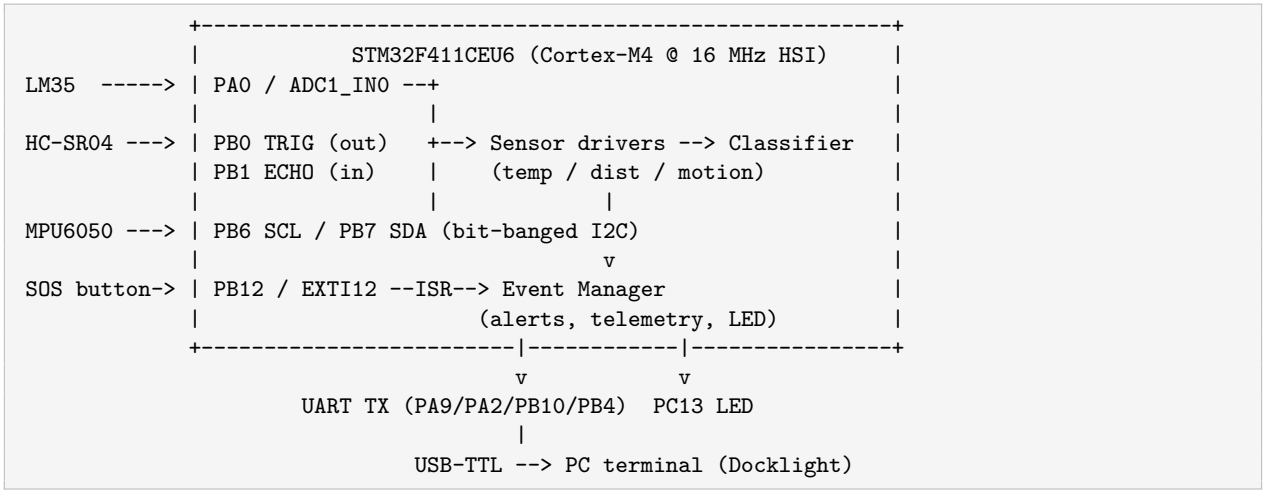
3.2 In Scope (as implemented)

- MPU6050, LM35 and HC-SR04 sensing

- Threshold-based classification in firmware
- UART alert and telemetry transmission
- LED indication and the PB12 hardware emergency interrupt

4 System Overview and Architecture

4.1 Block Diagram



4.2 Processing Pipeline

Step	Stage	Mechanism
1	Sense	ADC conversion (LM35); echo-pulse timing (HC-SR04)
2	Acquire	14-byte burst read of the MPU6050 over I^2C
3	Process	Averaging, scaling, baseline subtraction
4	Detect	Threshold comparison → `EventState`
5	Interrupt	PB12 falling-edge EXTI sets a critical motion state immediately
6	Respond	LED heartbeat or strobe on PC13
7	Communicate	ASCII alerts and `[DATA]` telemetry over UART

4.3 Main-Loop Timing

The application is a **cooperative super-loop** with a nominal 100 ms period (PRD requirement NF-01 allows up to 200 ms).

```
INIT  clock -> DWT -> GPIO -> EXTI -> UART -> ADC -> banner
      -> MPU6050 init + baseline
LOOP  (every ~100 ms)
        1. lm35_read_and_classify()
        2. hcsr04_read_and_classify()
        3. mpu6050_read_and_classify()  (or re-init if offline)
        4. transmit_alerts_and_telemetry()
        5. delay_ms(100)
```

Critical alerts are rate-limited to every 5th loop (≈0.5 s) and the normal heartbeat to every 10th loop (≈1 s), so a persistent fault does not flood the UART.

5 Hardware Design

5.1 Components

Component	Role	Interface
STM32F411CEU6 Black Pill	Main controller; Cortex-M4F, run at 16 MHz HSI	—
MPU6050 (GY-521)	3-axis accelerometer, 3-axis gyroscope, die temperature	I^2C
LM35	Analog temperature sensor, 10 mV/°C	ADC
HC-SR04	Ultrasonic distance sensor, 2–400 cm	GPIO + timing
USB-to-UART adapter (CP2102/CH340)	Bridge to the PC terminal	UART
On-board LED (PC13, active LOW)	Local status indication	GPIO
Push-button (PB12 to GND)	Manual emergency / SOS input	GPIO + EXTI

5.2 Pin Map (from `config.h`)

Signal	Pin	Function	Notes
LM35 output	PA0	ADC1_IN0	Analog mode, no pull
HC-SR04 TRIG	PB0	GPIO output	Push-pull, idle LOW
HC-SR04 ECHO	PB1	GPIO input, pull-down	5 V-tolerant pin, wired directly
MPU6050 SCL	PB6	Open-drain, pull-up	Bit-banged I^2C
MPU6050 SDA	PB7	Open-drain, pull-up	Bit-banged I^2C; AD0 to GND gives 0x68
UART TX (primary)	PA9	USART1 TX, AF7	Hardware UART
UART TX (mirror)	PA2	USART2 TX, AF7	Hardware UART
UART TX (mirror)	PB10, PB4	GPIO, bit-banged	139 cycles per bit
Status LED	PC13	GPIO output	Active LOW
Emergency input	PB12	EXTI12, falling edge, pull-up	NVIC priority 2

All pin numbers and thresholds are centralised in `config.h`; other code refers to them only through macros (PRD Modularity Rule 1).

5.3 Clocking

The firmware runs from the internal **16 MHz HSI** oscillator with all bus prescalers at 1 and zero flash wait states. The **DWT cycle counter** (`CYCCNT`) is the free-running time base. It gives cycle-exact delays (`delay_us`, `delay_ms`) and pulse-width measurement without a hardware timer.

6 Embedded Concepts Applied

Concept	Where used	What it demonstrates
Cortex-M4 core peripherals	DWT, CoreDebug, NVIC	Debug-unit cycle counter as a time base; interrupt priority control
Memory-mapped register I/O	All drivers	Direct MODER, AFR, BSRR, CR1/CR2, SR/DR access
GPIO	Trigger, echo, LED, button, bit-banged buses	Input, output, alternate and analog modes; atomic set/reset via BSRR
ADC	LM35	12-bit single conversion, 480-cycle sample time, software oversampling
Serial buses	MPU6050 (I^2C), alert link (UART)	Master protocol implementation, ACK handling, bus recovery
Interrupts (EXTI/NVIC)	PB12 emergency input	Short ISR, flag passing to the main loop, write-buffer drain
Timing	DWT delay and echo capture	Deterministic, hardware-independent timing
Fault tolerance	MPU6050 re-init, HC-SR04 timeouts	Graceful degradation; invalid readings never raise alerts
Observability	`volatile` globals	Live inspection in the Keil Watch window

7 Firmware Design

The main firmware is a single translation unit, `main.c`, with configuration in `config.h`.

7.1 Clock and Timing

`clock_init()` enables HSI, selects it as SYSCLK and clears the prescalers. `dwt_init()` enables the trace block and starts `CYCCNT`. `delay_us()` busy-waits on `CYCCNT` using unsigned subtraction, so it stays correct across counter wrap-around.

7.2 LM35 Temperature

- ADC1 is configured for 12-bit single conversion on channel 0 with the longest sample time (480 cycles), because the LM35 has a high source impedance.
- **32 conversions** are averaged per reading to suppress noise.
- Conversion: mV = raw × 3300/4095, then T = mV/10 (LM35 gives 10 mV/°C).
- Classification uses the rounded integer °C value.

7.3 HC-SR04 Distance

1. Drive TRIG HIGH for 15 µs (the sensor needs at least 10 µs), then LOW.
2. Wait for ECHO to rise, with a **35 ms timeout**. No echo means no target or no sensor.
3. Time how long ECHO stays HIGH using `CYCCNT`, with a **40 ms** ceiling.

4. $d_{\text{cm}} = t_{\text{us}}/58$ (sound travels about 58 µs per cm for the round trip).
5. Only values in **2–400 cm** are accepted. Anything else, and any timeout, gives `STATE_INVALID` with no alert (PRD NF-07).

7.4 Bit-Banged I²C

I²C is implemented in software on PB6/PB7 at roughly **50 kHz** (10 µs half-period). It provides:

- START/STOP generation and byte write/read with ACK/NACK,
- **clock-stretching support**: `i2c_scl_hi()` waits for the line to actually go high, with a timeout,
- **bus clear**: `i2c_bus_clear()` sends nine clock pulses plus a STOP to release a slave holding SDA low,
- register write, single-register read and burst read.

7.5 MPU6050 Driver

- **Auto-probe** of addresses 0x68 and 0x69, validated by reading `WHO_AM_I` (0x75) and rejecting 0x00 and 0xFF (a floating bus).
- **Initialisation:** wake from sleep, PLL clock with X-gyro reference, enable all axes, `SMPLRT_DIV`=0x07 (125 Hz), `CONFIG`=0x06 (DLPF ≈5 Hz), accelerometer ±2 g, gyroscope ±250 °/s, interrupts disabled.
- **Burst read:** all 14 bytes (accelerometer, temperature, gyroscope) in one transaction from 0x3B, combined into signed 16-bit values.
- **Baseline calibration:** 32 stationary samples, 10 ms apart, are averaged per axis. Live samples are compared against this baseline, so gravity and mounting orientation do not trigger false events.
- **Scaling:** ±2 g gives 16384 LSB/g and ±250 °/s gives 131 LSB/(°/s). The impact metric is $\max(|\Delta a_x|, |\Delta a_y|, |\Delta a_z|) \times 100/16384$, i.e. g×100 in integer arithmetic.
- **Self-healing:** on a read failure the state becomes `STATE_INVALID`, the bus is cleared and the sensor is re-initialised. If startup detection failed, the main loop retries `mpu6050_init()` every cycle.

7.6 Emergency Interrupt

PB12 is an input with pull-up, routed through `SYSCFG->EXTICR[3]` to EXTI12 and triggered on the **falling edge**. The ISR is short and:

1. clears the pending bit (write-1-to-clear) and reads `EXTI->PR` back to drain the write buffer, preventing spurious re-entry,
2. sets `emergency_exti_triggered`,
3. forces `state_motion = STATE_CRITICAL`, increments the alert counter and toggles the LED.

The ISR does no I²C and no UART (PRD NF-05); the main loop prints the alert.

7.7 UART Driver

Each character is sent on **four** outputs at 115200 baud, 8-N-1:

Output	Method
PA9	USART1 (hardware), BRR = 139 (16 MHz / 115200)
PA2	USART2 (hardware), BRR = 139
PB10, PB4	Software UART: start bit, 8 data bits LSB-first, stop bit, each 139 cycles long, timed from `CYCCNT`

The mirrored outputs let the team connect whichever USB-TTL adapter or header pin is available without reflashing. Hardware TXE waits have a 5000-iteration timeout so a disconnected port cannot hang the loop. `uart_send_string()` expands `\n` to `\r\n`.

7.8 Event Manager

- Emits `[ALERT]` lines for each sensor in `STATE_CRITICAL` (every 5th loop).
- Emits the `[INFO] SYSTEM | NORMAL` heartbeat plus a `[DATA]` telemetry line when nothing is critical (every 10th loop).
- Drives the LED: 10 Hz toggle on any critical state, 1 Hz heartbeat otherwise.

7.9 Debug Observability

Sensor values, raw counts, baselines, states and counters (`temperature`, `adc_mv`, `distance`, `echo_pulse_us`, `accel_x/y/z`, `max_delta_g_x100`, `critical_alert_count`, `mpu_id`, ...) are `volatile` globals that can be watched live in Keil µVision without UART.

8 Event Classification Logic

All thresholds live in `config.h`:

Sensor	Warning	Critical	Invalid
Temperature (LM35)	$T \geq 45$ °C	$T \geq 60$ °C	—
Distance (HC-SR04)	$d \leq 35$ cm	$d \leq 20$ cm	Timeout, or d outside 2–400 cm
Motion (MPU6050)	$\max \Delta \geq 0.80$ g	$\max \Delta \geq 1.50$ g	I^2C read failure

```
typedef enum {
    STATE_NORMAL   = 0,   /* within safe range            */
    STATE_WARNING  = 1,   /* approaching threshold        */
    STATE_CRITICAL = 2,   /* threshold exceeded -> alert  */
    STATE_INVALID  = 3    /* sensor offline / out of range */
} EventState;
```

Design rules applied:

- **Invalid readings never alert** (NF-07). A disconnected HC-SR04 reports `INVALID` but does not stop the temperature and motion checks.
- Only `STATE_CRITICAL` triggers an `[ALERT]`. `STATE_WARNING` is classified, but the current firmware prints no separate warning line.
- The motion metric is a **change from baseline**, not absolute acceleration, so it is orientation-independent.

9 UART Alert Protocol

Plain ASCII, one message per line, terminated by `\r\n`. Link parameters: **115200 baud, 8 data bits, no parity, 1 stop bit**.

```
[INFO]  SYSTEM   | BOOT     | Vehicle Event Monitor v1.0
[INFO]  SYSTEM   | INIT     | Calibrating sensors...
[INFO]  SYSTEM   | INIT     | MPU6050 baseline calibrated
[WARN]  SYSTEM   | INIT     | MPU6050 not detected
[ALERT] MOTION   | CRITICAL | Ax=+1.82g Ay=-0.12g Az=+0.03g
[ALERT] EXTI     | CRITICAL | Hardware Emergency Interrupt fired on PB12!
[ALERT] TEMP     | CRITICAL | T=73C (threshold=60C)
[ALERT] OBSTACLE | CRITICAL | d=14cm (threshold=20cm)
[INFO]  SYSTEM   | NORMAL   | All sensors OK
[DATA]  T=26C | d=31cm | Ax=+0.01g Ay=-0.02g Az=+0.00g
```

Message	When
`BOOT`	Once at power-up (confirms UART is alive)
`INIT`	Startup; reports MPU6050 calibrated or not detected
`ALERT MOTION`	Impact of at least 1.5 g from baseline
`ALERT EXTI`	SOS button on PB12 pressed
`ALERT TEMP`	Over-temperature
`ALERT OBSTACLE`	Obstacle within 20 cm
`NORMAL`	Heartbeat about every 1 s while nothing is critical
`DATA`	Telemetry block for the dashboard (`d=-cm` if invalid)

10 Testing and Validation

10.1 Standalone Test Programs

Each test is a separate `main()` that can be flashed in place of `main.c`.

- **`hrc_test.c`** – HC-SR04 bring-up. Counts valid versus total pings, exposes a stage diagnostic (`echo_diag_stage`: 1 trig sent, 2 echo high, 3 echo low, 4 OK, −1/−2 timeouts), toggles PC13 on each valid reading, and **auto-scans backup TRIG/ECHO pin pairs** (PB0/PB1, PA3/PA4, PA1/PA2 and swapped polarities) when no echo is seen.
- **`mpu6050_test.c`** – MPU6050 bring-up. **Auto-probes SCL/SDA pin pairs and I²C addresses**, reads `WHO_AM_I`, computes the acceleration magnitude (≈1.00 g at rest), and shows bus health on the LED (10 Hz when streaming, 1 Hz when searching).

10.2 Test Plan (from the PRD)

Test	Procedure	Pass criterion
LM35 normal	Read at room temperature	≈25 °C, `STATE_NORMAL`
LM35 response	Warm the sensor by hand	Reading rises
LM35 critical	Temporarily lower `TEMP_THRESHOLD_C` (e.g. 30)	`[ALERT] TEMP` fires
MPU6050 identity	Read `WHO_AM_I`	Valid ID (PRD expects 0x68)
MPU6050 stationary	Leave still for 30 s	Deltas ≈0; no false trigger
MPU6050 impact	Tap or shake the board	`[ALERT] MOTION` within one reading
HC-SR04 accuracy	Object at a known distance (e.g. 30 cm)	Within ±3 cm
HC-SR04 critical	Object at 10 cm	`[ALERT] OBSTACLE` fires
HC-SR04 timeout	Block or unplug ECHO	`STATE_INVALID`, no alert
EXTI	Press PB12 button	`[ALERT] EXTI` printed, LED strobes
Integration	All sensors for 60 s	No spurious alerts; sensors alert independently
Power cycle	Reset the board	Banner printed, baseline re-established, no boot alerts

10.3 Recorded Results

To be completed by the team. The repository contains no captured measurements, so no pass/fail results are claimed here. Record the bench results below before submission.

Test	Expected	Measured	Pass/Fail
LM35 at room temperature	≈25 °C		
HC-SR04 at 30 cm	30 ± 3 cm		
MPU6050 `WHO_AM_I`	0x68		
MPU6050 stationary 30 s	No false alerts		
Impact / shake	`[ALERT] MOTION`		
SOS button (PB12)	`[ALERT] EXTI`		
60 s integration run	No spurious alerts		

11 Implementation vs. Planned Design

The firmware departs from the PRD in several places; they are listed so the report matches the code.

Topic	PRD / plan	Implemented
MCU part	Plan: STM32F401CCUx; PRD: STM32F411CEU6	**STM32F411CEU6** (as in `config.h`)
System clock	84 MHz	**16 MHz HSI** (no PLL)
I^2C	Hardware I2C1 peripheral	**Bit-banged** I^2C on PB6/PB7 (more robust bring-up)
HC-SR04 capture	Timer input capture, TIM1_CH2	**GPIO polling with DWT cycle counter** on PB0/PB1
UART	USART2 on PA2	USART1 (PA9), USART2 (PA2) and **two software UART pins**, all mirrored
Arithmetic	Integer only (NF-04)	Integer for the impact metric; `float` for temperature, distance and display values
Module layout	Separate `.c/.h` per driver	**Single `main.c`** plus `config.h`
Threshold levels	Single critical threshold	Added **Warning** tier and an `INVALID` state
Extra feature	—	PB12 **EXTI emergency input**

12 Future Work

Implementation improvements to the existing system:

1. Move to the **84 MHz PLL** clock and hardware **I^2C1 / timer input-capture** peripherals, as in the PRD, to free the CPU.
2. Make UART transmission **interrupt- or DMA-driven** with a ring buffer so sensor reads are never delayed.
3. Replace floating-point maths with integer scaling, and split `main.c` into per-driver `.c/.h` modules with unit tests.
4. Make the PB12 path race-free: latch the event in a dedicated flag and debounce it in hardware or software.
5. Add a collision-versus-rough-road classifier based on event duration and multi-axis signature, and a watchdog timer for fault recovery. These are planned enhancements, not completed features.

13 Repository Structure

```
Vehicle_Event_Logging/
|-- main.c          Application firmware (drivers + main loop)
|-- config.h        Pins, thresholds, EventState enum
|-- hrc_test.c      HC-SR04 standalone test
|-- mpu6050_test.c  MPU6050 standalone test
|-- PRD_Vehicle_Event_Logging.md
|-- Vehicle_Event_Logging_Response_System_Plan.md / .xlsx
|-- team11.pptx     Presentation slides
|-- REPORT.md / REPORT.tex / REPORT.pdf
```

14 Build, Flash and Run

Toolchain: Keil µVision with the STM32F4 device pack (device `STM32F411CEUx`), CMSIS device header `stm32f411xe.h`.

1. Create a project for the STM32F411CEU6 and add `main.c` and `config.h`. To run a standalone test instead, add `hrc_test.c` or `mpu6050_test.c` in place of `main.c`.
2. Enable the standard C library (`stdio`) so `snprintf` works.
3. Build, then flash through ST-Link (SWD).
4. Connect a USB-to-UART adapter RX to **PA9** (or PA2 / PB10 / PB4) and GND to GND.
5. Open a terminal (Docklight / PuTTY) at **115200, 8-N-1**.
6. Reset the board. Expect the boot banner, then the heartbeat and telemetry lines once per second.

15 Conclusion

The project delivers a working three-sensor vehicle monitor on a bare-metal STM32F411: analog sensing (LM35 on the ADC), digital bus sensing (MPU6050 over a bit-banged I^2C master with bus recovery), timing-based sensing (HC-SR04 using the DWT cycle counter), an interrupt-driven emergency input, a four-way mirrored UART alert channel and local LED indication. It follows the PRD's classification model and message format and handles faults gracefully: invalid readings never raise alerts, and an offline MPU6050 is re-initialised automatically. Planned next steps are interrupt- or DMA-driven communication, integer-only arithmetic, event logging with timestamps, and recorded bench measurements.